Reinforcement Learning

Lecture 4 — Monte Carlo Methods

Przemysław Witaszczyk

Faculty of Physics, Astronomy and Applied Computer Science
Jagiellonian University

2026/2027

Where we are

Lecture 3 solved the MDP *with* the model: Dynamic Programming iterates the Bellman operators, using the known dynamics p in every *full-width* backup.

Here the model is dropped. The goal is unchanged — v_π, q_π — but they are now estimated from **sampled experience**. The move is a single substitution:

a value is an expectation of the return, $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$*;*
if that expectation cannot be computed, **estimate it by averaging sampled returns.**

This is the **bandit idea (L1) lifted to states and full returns**:

Bandit (L1)	$\longrightarrow$	**Monte Carlo (today)**
average sampled *rewards* $\to q_*(a)$	$\longrightarrow$	average sampled *returns* $\to v_\pi(s)$
explore/exploit on one slot	$\longrightarrow$	exploring starts / ε-soft (keep exploring)
(single policy)	$\longrightarrow$	behavior b vs target π, importance sampling

The skeleton $\text{New} \leftarrow \text{Old} + \alpha[\text{Target} - \text{Old}]$ returns with $\text{Target} = G_t$, a *complete sampled return*.

What Monte Carlo requires, and what it does not

Needs: only *experience* — sampled episodes

$$S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$$

from interaction or a simulator. No model p required.

Restriction: episodic tasks — every episode must *terminate*, so a complete return G_t exists.

Defining trait: MC learns from *complete* returns. The target is the *actual* return G_t — no bootstrapping (it never estimates a value from another estimate). TD (Lecture 5), by contrast, will bootstrap.

Plan: prediction of v_π comes first, then action values and the exploration problem, then control (finding a good policy), and last off-policy learning via importance sampling.

First-visit Monte Carlo prediction

Recall the definition of the value function: $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$, the *average return* obtained under the policy from state s. We cannot compute that average directly (it would need the model p), but we can **estimate** it the same way we estimate any average — by collecting samples of it and taking their mean.

Many episodes are run; each time s is entered, the return G_t that follows is one sample of $G_t \mid S_t = s$. Average those samples:

$$V(s) \leftarrow \text{ average of the returns } G_t \text{ observed after visits to } s.$$

The more samples we average, the closer $V(s)$ gets to the true mean $v_\pi(s)$ — this is the ordinary fact that a sample average approaches the true average as the number of samples grows (the *law of large numbers*).

"First-visit" (made precise in the algorithm box that follows) means: within one episode, use only the return after the *first* time we enter s. (Sutton & Barto, Ch. 5.1)

First-visit MC — the algorithm

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated

Initialize:

$V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$

$Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:

Append G to $Returns(S_t)$

$V(S_t) \leftarrow \text{average}(Returns(S_t))$

Reading the box. We walk each episode *backward* from the end, building up the return with $G \leftarrow \gamma G + R_{t+1}$ (this is the recursive form of the return, $G_t = R_{t+1} + \gamma G_{t+1}$, from Lecture 2 — computing it backward means each step costs one multiply-and-add). Whenever we reach a state we have not already seen *earlier in this same episode*, we record its return and fold it into that state's running average. (Sutton & Barto, Ch. 5.1)

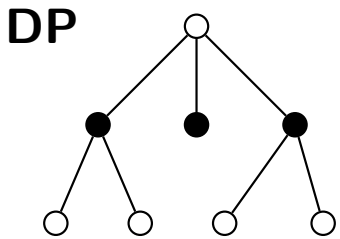
First-visit vs every-visit

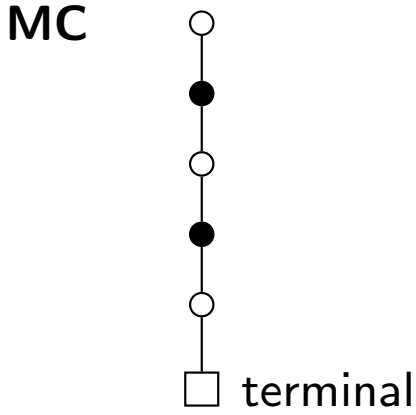
A single episode may visit the same state s several times. Two choices exist for which of those returns are averaged:

- **First-visit MC:** within each episode, keep only the return after the *first* time s is entered. Different episodes then give *separate, non-overlapping* returns — they share no rewards, so they are independent samples. An average of independent unbiased samples is itself **unbiased**: its expected value is exactly $v_\pi(s)$.
- **Every-visit MC:** average the returns after *every* visit, even several from one episode. Each episode contributes a number of returns that depends on the episode itself; this makes the estimate slightly **biased** when samples are few. The bias shrinks to zero as episodes accumulate — we say the estimate is *consistent* (it still reaches the right answer in the limit).

Both converge to $v_\pi(s)$; first-visit is easier to reason about, so it is our default. (Morales, Ch. 5; Sutton & Barto, Ch. 5.1; bias: Singh & Sutton 1996)

The Monte Carlo backup — sample, not full-width





DP backup (L3): the update covers one step and *all* successors, weighted by p — *full-width* — and it **bootstraps** (uses $v(s')$).

MC backup: follow *one* sampled trajectory all the way to termination; the target is the realized return G_t. No branching, no model, no bootstrapping.

This is the **full-vs-sample** axis from L3, now realized: MC replaces the model-weighted expectation with a sampled average, and the one-step bootstrap with the complete return.

Sampling replaces the model where p is unavailable

DP needed the full dynamics $p(s', r \mid s, a)$. For many episodic tasks p is impractical or impossible to write down, while the task is easy to *sample* (running it suffices): games, simulators, card games.

Morales' framing (Ch. 5): Monte Carlo is the method that *learns from complete, actual experience* — play a full episode to the end, tally the return obtained, and take the average of those returns. No model, no bootstrapping off a current estimate: only sampled returns, noisy but unbiased. It is the most direct response to the absence of a model — and the baseline against which TD (L5) is measured.

Example — Frozen Lake (our L2/L3 env). DP had to spell out the whole slippery-transition P dictionary; MC needs *none* of it — roll out episodes under π, average the returns. The cost: each episode must *end* before its return is available, and early estimates (few samples) vary considerably from run to run.

Forward reference: MC control recovers the optimal Frozen Lake policy that DP computes — now from samples alone, no model.

Worked estimate: first-visit MC on a three-state chain

The setup. A three-state episodic chain $s_1 \to s_2 \to s_3$ (with s_3 terminal). The agent moves right each step; the only reward is $+1$ on the step into the goal s_3. Discount $\gamma = 0.9$. One episode produces the states s_1, s_2 then termination, with rewards $R_1 = 0$ (the $s_1 \to s_2$ step) and $R_2 = +1$ (the $s_2 \to s_3$ step).

Compute the returns backward. The return is $G_t = R_{t+1} + \gamma G_{t+1}$, so working from the end of the episode toward the start (starting with $G = 0$ past the terminal state):

- Step $t = 1$ (we are in $s_2 = S_1$): $G_1 = R_2 + \gamma G_2 = 1 + 0.9 \cdot 0 = 1.0$.
- Step $t = 0$ (we are in $s_1 = S_0$): $G_0 = R_1 + \gamma G_1 = 0 + 0.9 \cdot 1.0 = 0.9$.

So this episode contributes the sample return 0.9 to state s_1 and 1.0 to state s_2. We add each to that state's running average:

$$V(s_1) \leftarrow \text{average of } \{0.9, \ \ldots\}, \qquad V(s_2) \leftarrow \text{average of } \{1.0, \ \ldots\}.$$

What "first visit" does. Suppose the dynamics were instead slippery, so an episode could revisit s_1: s_1, s_1, s_2, termination. We use s_1's return from its *first* occurrence only, so s_1 still receives exactly *one* sample from this episode. After many episodes the averages converge to v_π. This is the same update as L1's incremental average New $\leftarrow$ Old $+ \frac{1}{n}[\text{Target} - \text{Old}]$, now with Target $= G$ (a return) instead of a single bandit reward.

The need for *action*-value estimates without a model

With a model, a greedy policy reads off v:
$\pi'(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma v(s')]$ — but that needs p.

Without a model, $q_\pi(s, a)$ is estimated directly. Then

$$\pi'(s) = \arg\max_a q(s, a)$$

needs *no* model at all — this is why Lecture 2 carried both v and q. From here on, MC control estimates q.

MC evaluation of q_π: identical to the state-value case, but average the returns following first visits to the *pair* (s, a):

$$Q(s, a) \leftarrow \text{average}\{\, G_t : t \text{ is a first visit to } (s, a) \,\} \xrightarrow{\text{visits}\to\infty} q_\pi(s, a).$$

The maintaining-exploration problem

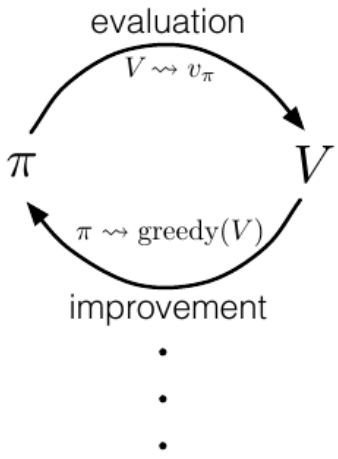
If π is **deterministic**, only *one* action is ever taken in each state. Then most pairs (s, a) are *never sampled*, so their $q(s, a)$ is never estimated and actions cannot be compared for improvement.

This is **L1's exploration–exploitation dilemma, lifted to state–action pairs**: to improve everywhere we must keep trying every action in every state.

Two remedies:

- **Exploring starts (ES):** begin episodes at *random* (s, a), every pair with nonzero probability.
- **Stochastic (ε-soft) policies:** the policy itself keeps exploring — every action has probability $\geq \varepsilon/|\mathcal{A}(s)|$.

Monte Carlo control = the GPI loop, with sampled evaluation



Recall **Generalized Policy Iteration (GPI)** from Lecture 3: alternate *evaluating* the current policy with acting *greedily* on that evaluation. Monte Carlo keeps the loop; evaluation is sampled, improvement reads q:

- **Evaluate:** estimate q_{π_k} by *averaging sampled returns* (instead of Dynamic Programming's full-width sweep over all successor states).
- **Improve:** $\pi_{k+1}(s) = \arg\max_a q_{\pi_k}(s, a)$ — greedy, no model.

The same loop, model-free: "evaluate $\leftrightarrow$ improve" is now "sample $\leftrightarrow$ improve."

The convergence argument (policy iteration), the two assumptions it requires, and their removal are treated below.

Recall: the policy-improvement theorem (L3)

Both control arguments below use one result from Lecture 3, restated here so that each is self-contained.

Policy-improvement theorem (L3)

Let π, π' be policies. **If** acting with π' for one step, then following π, is at least as good as π everywhere —

$$q_\pi\big(s, \pi'(s)\big) \;\geq\; v_\pi(s) \qquad \text{for all } s$$

— **then** π' is at least as good as π everywhere: $v_{\pi'}(s) \geq v_\pi(s)$ for all s (strictly better wherever the hypothesis is strict). *For stochastic π', read $q_\pi(s, \pi'(s))$ as $\sum_a \pi'(a \mid s)\, q_\pi(s, a)$ — the theorem holds unchanged.*

How we will use it: to prove a new policy π' improves, we need only check the *one-step* inequality $q_\pi(s, \pi'(s)) \geq v_\pi(s)$ — the theorem then upgrades it to a statement about the *full* values $v_{\pi'} \geq v_\pi$. The two MC-control slides that follow each do exactly this: establish the one-step inequality, then invoke the theorem.

Monte Carlo policy iteration — the formal argument

Run classical policy iteration with MC evaluation of *action* values:

$$\pi_0 \xrightarrow{\text{E}} q_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} q_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} q_*,$$

where $\xrightarrow{\text{E}}$ = MC evaluation, $\xrightarrow{\text{I}}$ = greedy improvement $\pi_{k+1}(s) = \arg\max_a q_{\pi_k}(s, a)$.

Improvement step. Establish the theorem's *one-step inequality* — read off q, no model needed:

$$\begin{aligned} q_{\pi_k}\big(s, \pi_{k+1}(s)\big) &= q_{\pi_k}\big(s, \arg\max_a q_{\pi_k}(s, a)\big) = \max_a q_{\pi_k}(s, a) \\ &\geq q_{\pi_k}\big(s, \pi_k(s)\big) = v_{\pi_k}(s). \end{aligned}$$

So $q_{\pi_k}(s, \pi_{k+1}(s)) \geq v_{\pi_k}(s)$ for all s; the policy-improvement theorem then gives $v_{\pi_{k+1}} \geq v_{\pi_k}$, i.e. $\pi_{k+1} \geq \pi_k$. With finitely many policies the strictly-improving chain reaches π_*.

Two assumptions make this exact, both unrealistic — we remove them shortly: **(A1)** infinitely many episodes per evaluation (so each q_{π_k} is exact); **(A2) exploring starts** (so every (s, a) is sampled).

Monte Carlo ES (Exploring Starts)

The algorithm under assumption (A2): **every** (s, a) **has nonzero probability of starting an episode**, so every action value is eventually estimated. It also folds evaluation and improvement together — improving at every visited state after each episode (not waiting for exact q).

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:
- $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
- $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
- $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever (for each episode):
- Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
- Generate an episode from S_0, A_0, following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
- $G \leftarrow 0$
- Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
  - $G \leftarrow \gamma G + R_{t+1}$
  - Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
    - Append G to $Returns(S_t, A_t)$
    - $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
    - $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

On Frozen Lake, MC-ES recovers the optimal policy that DP computes — from samples, with no model (about 10^6 episodes). (box: Sutton & Barto, Ch. 5.3)

Removing the two assumptions

(A1) Infinite episodes → episode-by-episode GPI. We need not evaluate q_{π_k} *exactly* before improving — the same issue arose in DP, and the same correction applies: truncate evaluation (L3's GPI / value-iteration idea). For MC the natural granularity is *one episode*: after each episode, update q at the visited pairs, then improve the policy there. We move *toward* q_{π_k}, not all the way. One caveat: whether this always converges is *an open question* — in Sutton's view one of RL's most fundamental (§5.3); the optimal policy is the only *stable* point, and he judges convergence to it very likely. *(This is the MC-ES algorithm box.)*

(A2) Exploring starts → keep exploring within the policy. ES is usually impossible (we rarely choose real start states). Instead require the policy itself to keep trying every action: a **stochastic, ε-soft** policy.

This mirrors L1: ES places exploration in the *start* state; ε-soft places it in the *action* choice — the same tension, lifted to (s, a). The cost: a soft policy learns only a *soft* π (on-policy) — unless we go off-policy (later this lecture), or decay $\varepsilon \to 0$ (a **GLIE** schedule: ε-soft MC then reaches π_*; the ceiling is a fixed-ε artifact).

On-policy control without ES: ε-soft policies

Keep the policy ε-**soft**: $\pi(a \mid s) \geq \dfrac{\varepsilon}{|\mathcal{A}(s)|}$ for all a (every action keeps some probability). The greedy step becomes ε-**greedy**: put $1 - \varepsilon + \varepsilon/|\mathcal{A}|$ on the best action, $\varepsilon/|\mathcal{A}|$ on each other.

On-policy first-visit MC control (for ε-soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ε-soft policy

$Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

On-policy: we evaluate and improve the *same* policy that generates the data. (Off-policy — two policies — comes later this lecture.) Whether improvement still holds is established in the theorem that follows. (Sutton & Barto, Ch. 5.4)

ε-greedy improvement — the theorem under exploration

Claim. If π' is ε-greedy w.r.t. q_π and π is any ε-soft policy, then $q_\pi(s, \pi'(s)) \geq v_\pi(s)$ — i.e. we again establish the theorem's *one-step inequality*, so the policy-improvement theorem gives $v_{\pi'} \geq v_\pi$. **Proof:**

$$
\begin{aligned}
q_\pi(s, \pi'(s)) &= \sum_a \pi'(a \mid s)\, q_\pi(s, a) = \frac{\varepsilon}{|\mathcal{A}|} \sum_a q_\pi(s, a) + (1 - \varepsilon) \max_a q_\pi(s, a) \\
&\geq \frac{\varepsilon}{|\mathcal{A}|} \sum_a q_\pi(s, a) + (1 - \varepsilon) \sum_a \underbrace{\frac{\pi(a \mid s) - \varepsilon/|\mathcal{A}|}{1 - \varepsilon}}_{\geq 0,\ \text{sums to 1}} q_\pi(s, a) \\
&= \frac{\varepsilon}{|\mathcal{A}|} \sum_a q_\pi(s, a) - \frac{\varepsilon}{|\mathcal{A}|} \sum_a q_\pi(s, a) + \sum_a \pi(a \mid s)\, q_\pi(s, a) = v_\pi(s).
\end{aligned}
$$

Two reusable steps: (i) max $\geq$ *any* convex average — the weights $\frac{\pi(a|s) - \varepsilon/|\mathcal{A}|}{1-\varepsilon}$ are ≥ 0 (π is ε-soft) and sum to 1; (ii) the $\varepsilon/|\mathcal{A}|$ terms **cancel**, rebuilding $\sum_a \pi(a|s) q_\pi = v_\pi$. L3's greedy improvement holds under the exploration constraint.

On-policy vs off-policy

Target policy π: the policy we want to learn about (e.g. the greedy/optimal one).
Behavior policy b: the policy that *generates* the data.

- **On-policy** ($b = \pi$): learn about the policy that is followed; simple, but it must remain exploratory *indefinitely* (ε-soft).
- **Off-policy** ($b \neq \pi$): explore with a soft b while learning about a sharp (even deterministic) π. More general, and the basis of much later machinery (Q-learning L5, deadly triad L9).

Coverage assumption: $\pi(a \mid s) > 0 \Rightarrow b(a \mid s) > 0$ — b must be able to produce every action π might take, or we could never observe it.

The idea of importance sampling

The difficulty is that the returns were generated by the *behavior* policy b, but we want the average return under the *target* policy π. A return that b produced often might be rare under π, and vice versa — so a plain average of the observed returns would estimate v_b, not v_π.

The repair — reweighting. Suppose a particular trajectory is *twice* as likely under π as under b. Then in a long run under b it occurs only about half as often as its frequency under π. To correct for that, we count its return *twice* as heavily. In general we weight each observed return by

$$(\text{weight}) = \frac{\text{probability of this trajectory under } \pi}{\text{probability of this trajectory under } b}.$$

This weight is called the **importance-sampling ratio**, written $\rho_{t:T-1}$. Trajectories with higher probability under π are up-weighted; those with lower probability are down-weighted (possibly to zero). The ratio is computed below and reduces to a product of per-step probability ratios.

Computing the ratio — the dynamics drop out

Probability of a trajectory tail. Starting from S_t, the agent alternately picks an action (probability set by the policy) and the environment picks the next state (probability set by the dynamics p). Multiplying these step probabilities:

$$\Pr\{A_t, S_{t+1}, A_{t+1}, \ldots, S_T \mid S_t\} = \prod_{k=t}^{T-1} \underbrace{\pi(A_k \mid S_k)}_{\text{policy picks } A_k} \underbrace{p(S_{k+1} \mid S_k, A_k)}_{\text{env. picks } S_{k+1}}.$$

Form the ratio. The behavior policy acts in the *same* environment, so its trajectory probability has the *same* p factors — only the policy factors differ. Dividing π's probability by b's, the dynamics factors are identical top and bottom and cancel term by term:

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)\, \cancel{p(S_{k+1} \mid S_k, A_k)}}{b(A_k \mid S_k)\, \cancel{p(S_{k+1} \mid S_k, A_k)}} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}.$$

This is why importance sampling is model-free. The ratio is a product of the two policies' probabilities for the actions we actually took — the environment dynamics p, which we do not know, have completely dropped out. (It is also why coverage is needed: an action π might take but b never takes never appears in the data, so the returns after it are never weighted and the estimate is biased.)

Two ways to use the weighted returns

Each first visit to s gives us a return G_t and its importance-sampling ratio $\rho_{t:T-1}$. Say there are N such first visits. The weighted returns $\rho_{t:T-1}G_t$ can be combined into one estimate in two natural ways:

$$\underbrace{V(s) = \frac{1}{N}\sum \rho_{t:T-1}\, G_t}_{\textbf{ordinary IS: divide by the count } N} \qquad \underbrace{V(s) = \frac{\sum \rho_{t:T-1}\, G_t}{\sum \rho_{t:T-1}}}_{\textbf{weighted IS: divide by the total weight}}$$

So **ordinary** is a straight average of the weighted returns, while **weighted** is a true weighted average (same weights $\rho_{t:T-1}$ on top and bottom). They behave differently:

	bias	**variance (run-to-run spread)**
Ordinary	none (unbiased)	high — can even be infinite
Weighted	some, but shrinks to 0	much lower; the practical choice

The same unbiased-vs-consistent distinction as first- vs every-visit; each claim is proved or measured below.

(a) Why ordinary importance sampling is unbiased

We want to show that the weighted return $\rho_{t:T-1}\, G$ has, on average under the behavior policy b, exactly the value we want, $v_\pi(s)$. "On average" means an expectation, and an expectation is a sum over all possible outcomes (here, trajectories τ), each weighted by how likely it is. Under b, trajectory τ occurs with probability $\Pr_b(\tau \mid s)$. So:

$$\begin{aligned}
\mathbb{E}_b\big[\, \rho_{t:T-1}\, G(\tau) \mid S_t = s \,\big] &= \sum_\tau \Pr_b(\tau \mid s)\; \rho_{t:T-1}\; G(\tau) && \text{(prob. } \times \text{ value)} \\
&= \sum_\tau \Pr_b(\tau \mid s)\; \frac{\Pr_\pi(\tau \mid s)}{\Pr_b(\tau \mid s)}\; G(\tau) && \text{(ratio} = \Pr_\pi / \Pr_b\text{)} \\
&= \sum_\tau \Pr_\pi(\tau \mid s)\; G(\tau) && (\Pr_b \text{ cancels}) \\
&= \mathbb{E}_\pi\big[\, G(\tau) \mid S_t = s \,\big] = v_\pi(s). && (= \text{the expectation und}
\end{aligned}$$

The middle step carries the argument: multiplying by the ratio replaces each trajectory's behavior-probability with its *target*-probability, turning an average over b's data into the average required (over π). Every off-policy method that reweights data uses this identity — it returns in Lectures 6, 9, 11 and 12.

(b) Why weighted importance sampling is biased

The bias is most easily seen in the extreme case of a *single* observed return G with ratio ρ (taking $\rho > 0$ — a $\rho = 0$ trajectory contributes no weighted estimate at all). The weighted estimator divides by the total weight — which here is just ρ — so the ρ cancels completely:

$$V(s) = \frac{\rho\, G}{\rho} = G.$$

The estimate is G itself, the return as produced by b. Its average value is therefore

$$\mathbb{E}_b[V(s)] = \mathbb{E}_b[G \mid S_t = s,\ \rho > 0],$$

an average under the *behavior* policy's probabilities — in general not $v_\pi(s)$. So with few samples the weighted estimate is **biased**.

Compare **ordinary** IS on that same single return: it gives ρG, whose average is $v_\pi(s)$ by part (a) — **unbiased**, every time.

As we gather more returns, the weighted estimate's bias shrinks to zero, while its run-to-run spread is far smaller than that of ordinary IS; the two figures below quantify the difference.

Ordinary vs weighted — and infinite variance

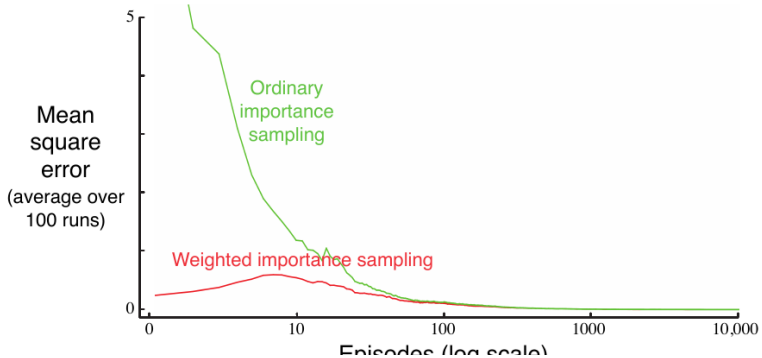


Fig 5.3: weighted IS has far lower error early on (Blackjack state).

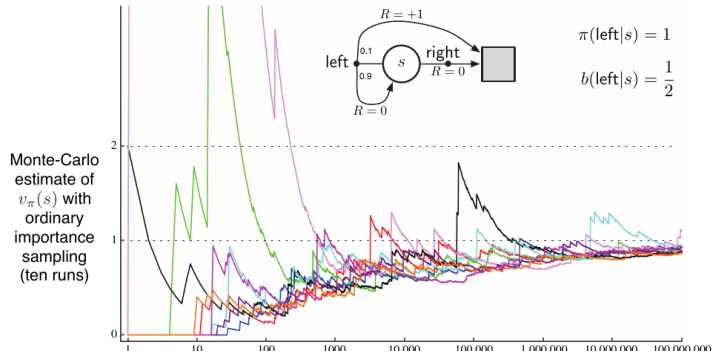


Fig 5.4: a one-state MDP where *ordinary* IS has literally **infinite variance** — estimates never settle, even after 10^8 episodes.

The single-state example ($\pi(\text{left}) = 1$, $b(\text{left}) = \frac{1}{2}$) lets the infinite variance be computed exactly, not merely observed. The variance is *proved* infinite below — weighted IS is the practical default. (Sutton & Barto, Ch. 5.5)

(Worked) Why the variance is infinite

The MDP (undiscounted, $\gamma = 1$)**.** From s: `left` returns to s w.p. 0.9 (reward 0) or terminates w.p. 0.1 (reward $+1$); `right` terminates (reward 0). Target $\pi(\text{left}) = 1$; behavior $b(\text{left}) = b(\text{right}) = \frac{1}{2}$. True value $v_\pi(s) = 1$.

Reduce to one quantity. Since $\mathrm{Var}[X] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$ and the mean is finite ($= 1$), the variance is infinite *iff* $\mathbb{E}[X^2] = \infty$, where $X = \left(\prod_t \frac{\pi(A_t|S_t)}{b(A_t|S_t)}\right) G_0$.

Enumerate trajectories. Any episode ending in `right` has ratio 0 (target never goes right); ignore it. A length-k episode with nonzero ratio is $k-1$ self-loops then a final `left`, return $G_0 = 1$:

$$\mathbb{E}[X^2] = \sum_{k=1}^{\infty} \underbrace{\left(0.9 \cdot \tfrac{1}{2}\right)^{k-1}\left(0.1 \cdot \tfrac{1}{2}\right)}_{\Pr_b(\text{episode})} \cdot \underbrace{\left(\frac{1}{1/2}\right)^{2k}}_{(\text{ratio})^2} = 0.1 \sum_{k=1}^{\infty} (1.8)^{k-1} \cdot 2 = \infty.$$

The ratio^2 grows like 4^k while the trajectory probability only decays like 0.45^k — their product grows like 1.8^k, so the series **diverges**. Ordinary IS is unbiased yet has infinite variance here; weighted IS gives exactly 1 after the first valid episode.

Incremental implementation (weighted IS)

The returns are not stored individually; the update is incremental, generalizing L1's incremental mean to a *weighted* average. With weights $W_n = \rho^{(n)}$ and cumulative weight $C_n = \sum_{k \le n} W_k$:

$$V_{n+1} = V_n + \frac{W_n}{C_n}\big[\, G_n - V_n \,\big], \qquad C_{n+1} = C_n + W_{n+1}.$$

Same skeleton New $\leftarrow$ Old $+ \alpha$[Target $-$ Old] from L1 — here the step size is the *weight fraction* W_n / C_n (it reduces to $1/n$ when all weights are 1, recovering the plain mean).

Why W accumulates step-by-step. The ratio *factorizes* — read it off the product from the IS slide:

$$\rho_{t:T-1} = \frac{\pi(A_t|S_t)}{b(A_t|S_t)} \cdot \rho_{t+1:T-1} \quad \Longrightarrow \quad W \leftarrow W \cdot \frac{\pi(A_t|S_t)}{b(A_t|S_t)} \text{ per step.}$$

This is why the algorithm boxes update W multiplicatively as they walk the episode backward — and why a single off-target action ($\pi(A_t|S_t) = 0$) sends $W \to 0$ and zeroes every earlier step of the backward pass. (This recursive ratio returns in off-policy methods that reweight returns — Lectures 6, 9, 11 and 12; Q-learning, L5, needs no ratio.)

Off-policy MC: prediction and control

Off-policy MC prediction (policy evaluation) for estimating $Q \approx q_\pi$

Input: an arbitrary target policy π
Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
  $Q(s,a) \in \mathbb{R}$ (arbitrarily)
  $C(s,a) \leftarrow 0$

Loop forever (for each episode):
  $b \leftarrow$ any policy with coverage of π
  Generate an episode following b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  $W \leftarrow 1$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$, while $W \neq 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$
    $W \leftarrow W \frac{\pi(A_t|S_t)}{b(A_t|S_t)}$

Prediction: evaluate q_π from data generated by b.

Off-policy MC control, for estimating $\pi \approx \pi_*$

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
  $Q(s,a) \in \mathbb{R}$ (arbitrarily)
  $C(s,a) \leftarrow 0$
  $\pi(s) \leftarrow \operatorname{argmax}_a Q(s,a)$ (with ties broken consistently)

Loop forever (for each episode):
  $b \leftarrow$ any soft policy
  Generate an episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  $W \leftarrow 1$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$
    $\pi(S_t) \leftarrow \operatorname{argmax}_a Q(S_t, a)$ (with ties broken consistently)
    If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)
    $W \leftarrow W \frac{1}{b(A_t|S_t)}$

Control: b soft (explores), π greedy (target).

Walking each episode backward, the per-step weight builds up as $W \leftarrow W \cdot \frac{1}{b(A_t \mid S_t)}$ (the target is greedy, so $\pi(A_t|S_t) = 1$ as long as the trajectory follows it). Early exit: the moment the trajectory takes an action the greedy π would *not*, $\pi(A_t|S_t) = 0$ sets the weight to zero — so off-policy MC learns only from episode *tails* consistent with π. (Sutton & Barto, Ch. 5.6–5.7)

Off-policy variants & a forward reference

Mention only (Sutton §5.8–5.9): *discounting-aware* and *per-decision* importance sampling further reduce variance by exploiting the structure of the return; the ideas matter, but the course does not treat them further.

Forward reference to Lecture 9. The **off-policy** component of the **deadly triad** has now been introduced; the display below dates the other two:

$$\begin{aligned} &\textit{off-policy}\ (\text{today}) \ + \ \textit{bootstrapping}\ (\text{L5}) \\ +\ &\textit{function approximation}\ (\text{L8}) \ = \ \text{instability}\ (\text{L9}). \end{aligned}$$

Tabular off-policy MC is *stable* — it neither bootstraps nor approximates. Instability arises only when all three are combined. (L3's $(I - \gamma P_\pi)$ remark gave the first pointer to L9.)

Monte Carlo among DP and TD

	backup width	**target**
DP (L3)	full-width (all successors)	bootstrap $r + \gamma v(s')$
Monte Carlo (L4)	sample (one trajectory)	full return G_t (no bootstrap)
TD (L5)	sample (one step)	bootstrap $R_{t+1} + \gamma V(S_{t+1})$

Monte Carlo: **model-free**, **sample** backup, **no bootstrap**, on- or off-policy. It is still **GPI** — evaluation changed from *compute the expectation* to *sample it*, and improvement reads q.

The cost of "no bootstrap" is twofold: each episode must end before learning (episodic tasks only), and the variance is high. Temporal-difference learning (Lecture 5) removes the first and sharply reduces the second, at the price of bias.

Bridge to Lecture 5: bootstrapping the return

Monte Carlo waits for the episode to *end* before it can learn. **Temporal- Difference** learning replaces the tail of G_t with a *bootstrapped estimate*:

$$\underbrace{G_t}_{\text{MC target}} = R_{t+1} + \gamma R_{t+2} + \cdots \longrightarrow \underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}}.$$

$\Rightarrow$ learn from *incomplete* episodes, *online*, step by step — combining DP's bootstrap with MC's sampling.

The Target takes a third form:

bandit reward R $\rightarrow$ MC return G_t $\rightarrow$ TD bootstrap $R_{t+1} + \gamma V(S_{t+1})$.

Same skeleton New $\leftarrow$ Old $+ \alpha$[Target $-$ Old] throughout.

Summary — Lecture 4

- **Drop the model:** estimate $v_\pi(s) = \mathbb{E}_\pi[G_t | S_t{=}s]$ by **averaging sampled returns** (a sample average approaches the true average). First-visit vs every-visit.
- **Exploration problem** lifted to state–action pairs (s, a): **exploring starts** or ε-**soft** policies; the ε-greedy improvement proof re-establishes the hypothesis of L3's theorem under exploration.
- **On- vs off-policy**: behavior b vs target π; the **importance-sampling ratio** (dynamics cancel $\Rightarrow$ model-free); the **reweighting identity** $\mathbb{E}_b[\rho G] = v_\pi$.
- **Ordinary vs weighted IS**: unbiased/high-variance vs biased/low-variance; weighted is the default.
- **MC control = GPI** with sampled evaluation.

Recurring devices: the update skeleton New $\leftarrow$ Old $+ \alpha$[Target $-$ Old], GPI, the full-vs-sample axis, and exploration–exploitation lifted to (s, a).
Lecture 5 treats Temporal-Difference learning: the return is bootstrapped, and learning proceeds online from incomplete episodes.